

BUILD-API.md

3. Conventions REST

Version : V1

Statut : Référentiel officiel des conventions REST de Vitala

3.1 Objectif

Cette section définit les conventions REST applicables à toutes les API de Vitala.

Son objectif est de garantir :

- une architecture homogène ;
- des API prévisibles ;
- une excellente expérience développeur ;
- une maintenance facilitée ;
- une compatibilité durable entre les différents clients (Web, Mobile, Admin et services internes).

Toutes les nouvelles API doivent respecter ces conventions.

3.2 Principes REST

Les API de Vitala suivent les principes REST.

Chaque ressource est identifiée par une URI unique et manipulée au moyen des méthodes HTTP standards.

Les API doivent être :

- stateless ;
 - orientées ressources ;
 - uniformes ;
 - sécurisées ;
 - documentées.
-

3.3 Convention de nommage

Les ressources utilisent :

- des noms explicites ;

- des noms au pluriel ;
- des caractères minuscules ;
- le séparateur `-` uniquement lorsque nécessaire.

Exemples :

```
/users
/flashs
/missions
/media
/notifications
/settings
```

Les verbes ne doivent jamais apparaître dans l'URL.

Incorrect :

```
/createFlash
/getUser
/deleteMission
```

Correct :

```
POST /flashs
GET /users/{id}
DELETE /missions/{id}
```

3.4 Méthodes HTTP

Les méthodes HTTP sont utilisées selon leur sémantique.

Méthode	Usage
GET	Lecture
POST	Création
PUT	Remplacement complet
PATCH	Mise à jour partielle
DELETE	Suppression

Aucune méthode ne doit être détournée de son usage.

3.5 Structure des URI

Les URI suivent la structure :

```
/api/v1/{resource}  
/api/v1/{resource}/{id}  
/api/v1/{resource}/{id}/{sub-resource}
```

Exemples :

```
/api/v1/users  
/api/v1/users/{id}  
/api/v1/flashses  
/api/v1/flashses/{id}  
/api/v1/missions/{id}/participants
```

3.6 Paramètres

Les paramètres sont répartis selon leur nature :

Path Parameters

Identifient une ressource.

Exemple :

```
/users/{userId}
```

Query Parameters

Utilisés pour :

- filtrer ;
- trier ;
- rechercher ;
- paginer.

Exemple :

```
/flashses?page=2&limit=20&sort=created_at
```

Les données métier ne doivent jamais être transmises dans l'URL.

3.7 Corps des requêtes

Les requêtes utilisent le format JSON.

Le corps doit :

- être valide ;
 - respecter le schéma attendu ;
 - être entièrement validé avant traitement.
-

3.8 Corps des réponses

Toutes les réponses utilisent JSON.

Une réponse réussie doit rester :

- cohérente ;
- documentée ;
- prévisible.

Les structures de réponse doivent être uniformes dans toute la plateforme.

3.9 Pagination

Les collections volumineuses doivent être paginées.

Les paramètres standards sont :

- page
- limit

Les réponses doivent inclure :

- nombre total d'éléments ;
 - page courante ;
 - nombre total de pages ;
 - taille de page.
-

3.10 Filtrage

Les filtres utilisent les paramètres de requête.

Exemple :

```
/status=active  
/category=health
```

Les filtres doivent être combinables.

3.11 Tri

Le tri est réalisé via un paramètre dédié.

Exemple :

```
sort=created_at  
sort=-created_at
```

Le signe "-" indique un ordre décroissant.

3.12 Recherche

Les recherches textuelles utilisent :

```
search=mot
```

Les recherches doivent être sécurisées contre les injections et optimisées pour les performances.

3.13 Codes HTTP

Les API utilisent exclusivement les codes HTTP appropriés.

Exemples :

- 200 OK
- 201 Created
- 204 No Content
- 400 Bad Request
- 401 Unauthorized
- 403 Forbidden
- 404 Not Found
- 409 Conflict
- 422 Unprocessable Entity
- 429 Too Many Requests
- 500 Internal Server Error

Les codes doivent refléter précisément le résultat de la requête.

3.14 Sécurité

Toutes les API doivent :

- utiliser HTTPS ;
 - vérifier l'authentification ;
 - appliquer les autorisations ;
 - respecter les politiques RLS ;
 - enregistrer les actions sensibles.
-

3.15 Documentation

Chaque endpoint doit documenter :

- son objectif ;
 - sa méthode HTTP ;
 - son URI ;
 - ses paramètres ;
 - ses réponses ;
 - ses erreurs ;
 - les permissions requises.
-

3.16 Bonnes pratiques

Les API doivent :

- rester simples ;
 - éviter les imbrications excessives ;
 - limiter les traitements dans une seule requête ;
 - retourner uniquement les données nécessaires ;
 - rester compatibles avec les évolutions futures.
-

3.17 Conclusion

Les conventions REST de Vitala garantissent une architecture d'API uniforme, lisible et évolutive.

Elles assurent une expérience cohérente pour les applications clientes, les développeurs et les intelligences artificielles, tout en facilitant la maintenance et l'évolution de la plateforme.